

Deep Learning Pasta Identifier

This notebook will be generating and exporting an AI model for identifying pasta shapes (from the top 8 most popular). fastai will be utilised, as a high-level deep learning library, to assist with my learning. The exported model will then be used in a gradio web application hosted via huggingface.

```
In [21]: %%capture
!pip install fastbook
!pip install duckduckgo-search
```

```
In [22]: from fastbook import *
from duckduckgo_search import DDGS
from fastcore.all import *
from fastdownload import download_url
from fastai.vision.all import *
from fastai.vision.widgets import import *

def search_images(keywords, max_images=100):
    """This function uses the duckduckgo API to return links to a specified number of images 'max_images' using 'keyw
    return L(DDGS().images(keywords, max_results=max_images)).itemgot('image')
```

We have a function that will search duckduckgo. For the top 10 most mentioned pasta shapes online, we will download 100 photos to train/verify the model with.

```
In [23]: # define the pasta types the AI model will detect
pasta_types = ['spaghetti', 'penne', 'fusilli', 'farfalle', 'fettuccine', 'macaroni', 'orecchiette', 'gnocchi']
path = Path('pastas')

# download the 100 images locally
for pasta in pasta_types:
    dest = (path/pasta)
    dest.mkdir(exist_ok=True, parents=True)
    download_images(dest, urls=search_images(f'{pasta} pasta'))
```

It is important to remove any failed images to prevent them from breaking the model.

```
In [24]: # we will remove the failed images
failed = verify_images(get_image_files(path))
failed.map(Path.unlink)
len(failed) # to check how many failed (out of 800)
```

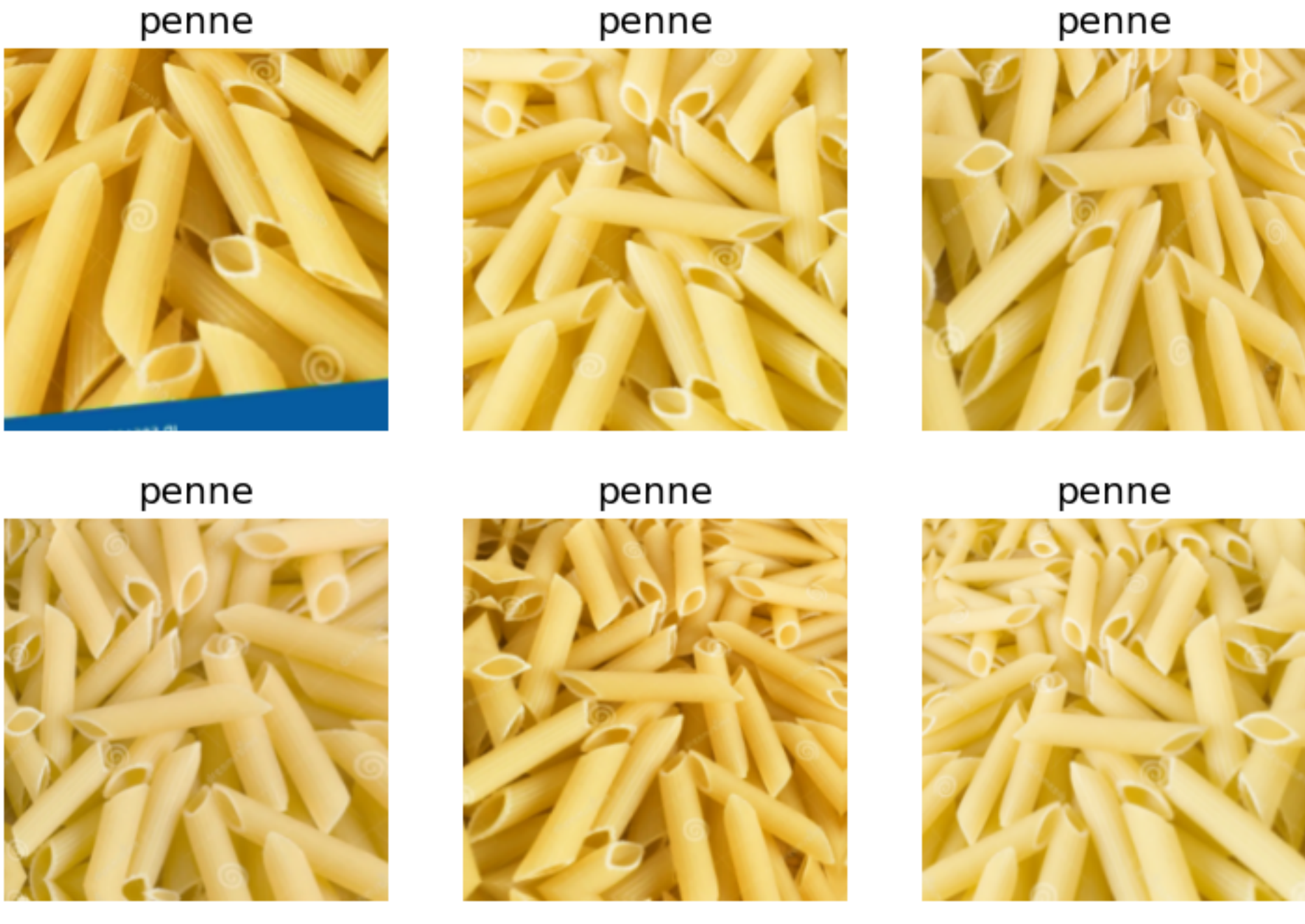
Out[24]: 10

To fit the model, we use DataLoaders - an object containing the training set and validation set. DataBlock is a flexible API from fastai to to define a data pipeline (how data is loaded, processed and split). DataLoaders is the object responsible for feeding batches of data to the model during training.

- `blocks` defines input type and output type- we are mapping images to categories
- `get_items` defines how to find all inputs to our model
- `splitter` how to split between training and validation
- `get_y` the name of the parent of each file (i.e., the name of the folder they're in)
- `item_tfms` we use `RandomResizedCrop` to take a different part of each image on each epoch to improve the model without increasing the dataset. Because we are feeding the model batches of images (tensors) we need to make sure the images are the same size.
- `batch_tfms` we augment (create random variations of input data) batches of images, leveraging parallel processing capabilities of the GPUs. Again, improving the model without increasing the dataset.

```
In [25]: # now we will create the data loaders object for the model
dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=RandomResizedCrop(224, min_scale=0.3), # model benefits from different parts of image
    batch_tfms=aug_transforms() # to improve the model without increasing the size of the dataset
).dataloaders(path)
```

```
In [26]: # we can inspect our training photos
dls.train.show_batch(max_n=6,unique=True)
```



Now we are ready to train our model by creating a learner object. `vision_learner` will set up a learner for image classification and we will use `resnet18` which a pretrained model with 18 layers designed for image classification. We will use `fine_tune` to adapt the model for our purposes and `error_rate` as the metric to evaluate its performance on each epoch (2).

```
In [27]: import warnings
warnings.filterwarnings("ignore", message="Palette images with Transparency expressed in bytes should be converted
learn = vision_learner(dls, resnet18, metrics=error_rate)
learn.fine_tune(2)
```

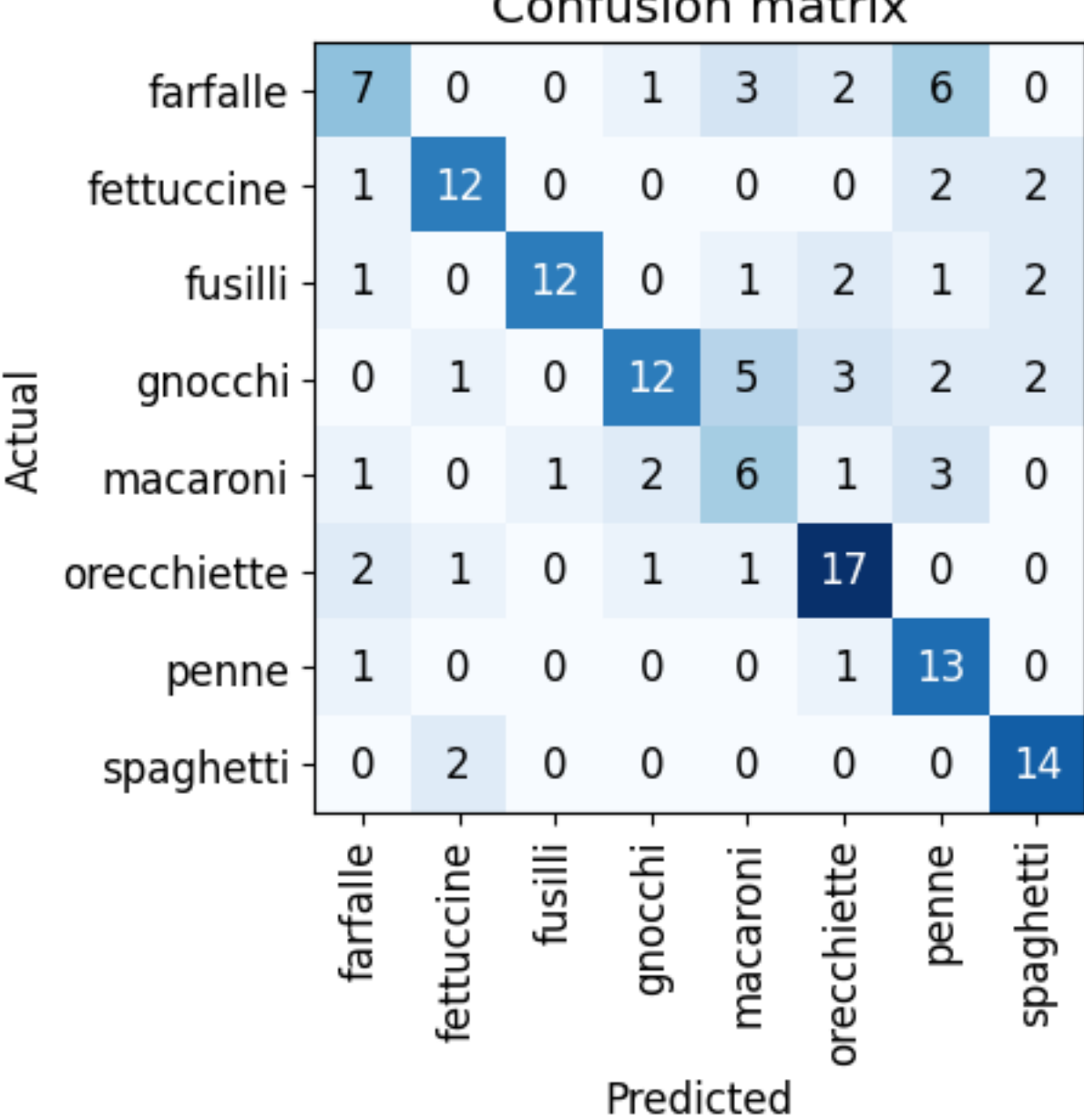
epoch	train_loss	valid_loss	error_rate	time
0	2.906840	1.947798	0.598639	02:29

epoch	train_loss	valid_loss	error_rate	time
0	1.706324	1.207722	0.435374	03:18
1	1.358913	0.982163	0.367347	03:16

Now that we have a model, we want to ensure it is accurate. We will do this in two ways.

- Firstly, we will check the confusion matrix to see if we need to improve our data.
- Then, we will use `ImageClassifierCleaner` to remove any incorrect or unsuitable images from the dataset.

```
In [28]: interp = ClassificationInterpretation.from_learner(learn)
interp.plot_confusion_matrix()
```



```
In [29]: cleaner = ImageClassifierCleaner(learn)
cleaner
```

VBox(children=(Dropdown(options=('farfalle', 'fettuccine', 'fusilli', 'gnocchi', 'macaroni', 'orecchiette', 'p...

```
In [30]: # remove based on ImageClassifierCleaner
for idx in cleaner.delete(): cleaner.fns[idx].unlink()
for idx, cat in cleaner.change(): shutil.move(str(cleaner.fns[idx]), path/cat)
```

The model is performing well considering the small nuances between different types of pasta (take fettuccine and spaghetti for example). It has also been trained on a relatively small set of data. Using `ImageClassifierCleaner` I have been able to remove some erroneous images that were classed incorrectly; however, in general the dataset is good quality.

Upon analysis of the confusion matrix we can see some issues. There is generally a large spread of errors, with a particular concentration where farfalle is being confused with penne. We will add more farfalle photos to the dataset to improve its predictions and fine-tune again. I also notice that macaroni is being poorly identified so we will add more photos of that as well.

```
In [35]: # this code adds extra photos but ensure we don't re-add the same photos
for pasta in ['macaroni', 'farfalle']:
    dest = path/pasta
    existing = set([f.name for f in dest.iterdir() if f.is_file]) # 0(1) look up
    new_image_urls = search_images(f'{pasta} pasta', max_images=150) # make new search (most will be the same)
    new_images = []
    for url in new_image_urls:
        image_name = url.split('/')[-1]
        if image_name not in existing: # make sure photo not already included
            new_images.append(url)
    download_images(dest, urls=new_images) # add new images
    failed = verify_images(get_image_files(dest))
    failed.map(Path.unlink)
    print(f'{len(failed)} photos failed for {pasta}') # to check how many failed
```

17 photos failed for macaroni
2 photos failed for farfalle

```
In [36]: # we can check the number of images in each folder now to make sure
for label in path.ls():
    print(label, len(get_image_files(label)))
```

pastas/penne 95
pastas/spaghetti 96
pastas/gnocchi 95
pastas/farfalle 504
pastas/macaroni 578
pastas/orecchiette 94
pastas/fusilli 90
pastas/fettuccine 93

Now that we have added more photos for farfalle and macaroni, we can retrain the model and check for improvements.

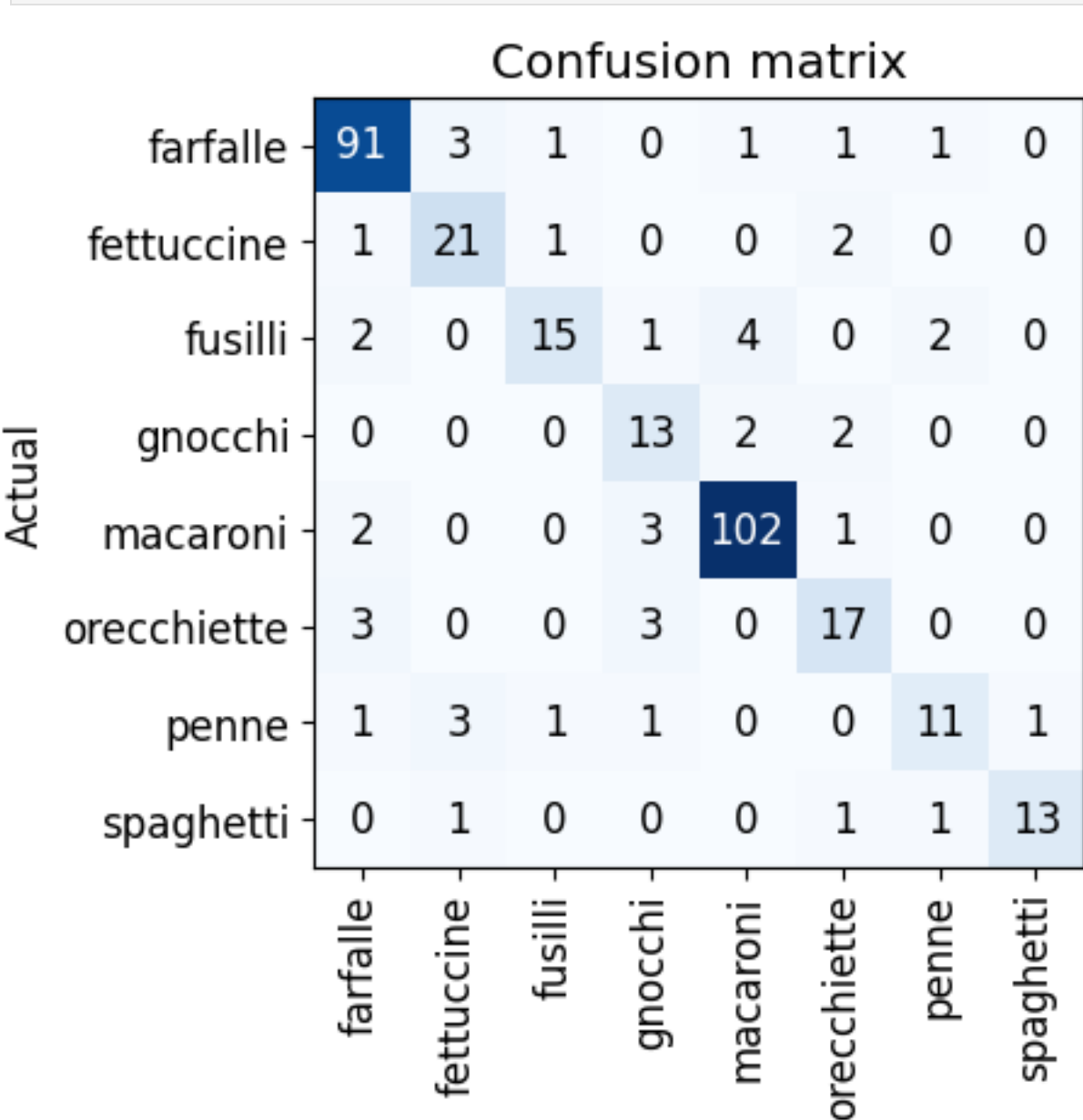
```
In [43]: dls = DataBlock(
    blocks=(ImageBlock, CategoryBlock),
    get_items=get_image_files,
    splitter=RandomSplitter(valid_pct=0.2, seed=42),
    get_y=parent_label,
    item_tfms=RandomResizedCrop(224, min_scale=0.3),
    batch_tfms=aug_transforms()
).dataloaders(path)
```

```
In [45]: learn = vision_learner(dls, resnet18, metrics=error_rate)
learn.fine_tune(2)
```

epoch	train_loss	valid_loss	error_rate	time
0	2.323375	1.063918	0.328267	05:18

epoch	train_loss	valid_loss	error_rate	time
0	0.993484	0.598862	0.206687	07:21
1	0.761495	0.467821	0.139818	07:14

```
In [46]: interp = ClassificationInterpretation.from_learner(learn)
interp.plot_confusion_matrix()
```



The model still has a 14% error rate, but overall it has seen a massive improvement by only increasing the volume of data in two categories. While further improvements to the data could likely be made, overall this is a suitable result to be exported. Since pasta shapes can be difficult to identify - even for humans - and this is a very basic model doing only 2 epochs, this is a fantastic result. Increasing the dataset or improving the quality of the images or running on a more layered pre-trained neural network or completing more epochs would improve the model. However, the fitting is already taking extensive time, so due to limited GPU resources, the overall result is adequate. We can use this in the web application and people can view the percentage likelihoods of each inference.

```
In [47]: # export the model
learn.export()
```

In []: